

Bouira University

Chapter 1: Introduction to Object-Oriented Programming & Java

Course for second-year (L2) Computer Science

Department of Computer Science.

Bouira university

Presented by

Dr S.BENBATATA

Programmation Orientée Objet

POO Coures

Credits: 5

Coefficient: 3

Introduction to Object-Oriented Programming



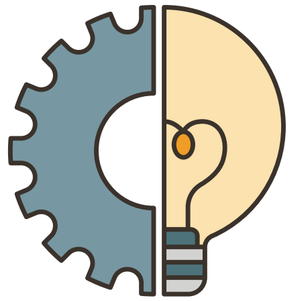
problem



programming paradigm

Treatment

Var



Prediction

Programming Paradigms

A programming paradigm is a style or way of writing code.

It gives you a set of rules and ideas that shape how you think about a problem and how you structure your program.

The types of praradigms:

- Procedural Programming
- Object-Oriented Programming
- Declarative Programming





- **Procedural Programming**

  Follows a step-by-step sequence of instructions. It's like a recipe – you do this, then that, then finish.
  For example : C, Pascal.





- **Procedural Programming**

  Follows a step-by-step sequence of instructions. It's like a recipe – you do this, then that, then finish.
  For example : C, Pascal.

- **Object Oriented Programming**

  OOP organizes code into objects that encapsulate data and methods
  Examples: Java, C++, Python



- **Procedural Programming**

  Follows a step-by-step sequence of instructions. It's like a recipe – you do this, then that, then finish.
  For example : C, Pascal.

- **Object Oriented Programming**

  OOP organizes code into objects that encapsulate data and methods
  Examples: Java, C++, Python

- **Declarative programming**

  focuses on what the program should do, not how to do it.

Declarative programming

Functional Programming:

Focuses on functions and avoids changing data. It's like math , you give input-->get output
For example: Lisp.

Logic Programming:

Is based on **facts** and **rules**. You give the program a set of truths, and it uses logic to find answers on its own.
For example:Prolog

Domain specific language:

Is a programming language made for a specific type of task or field.

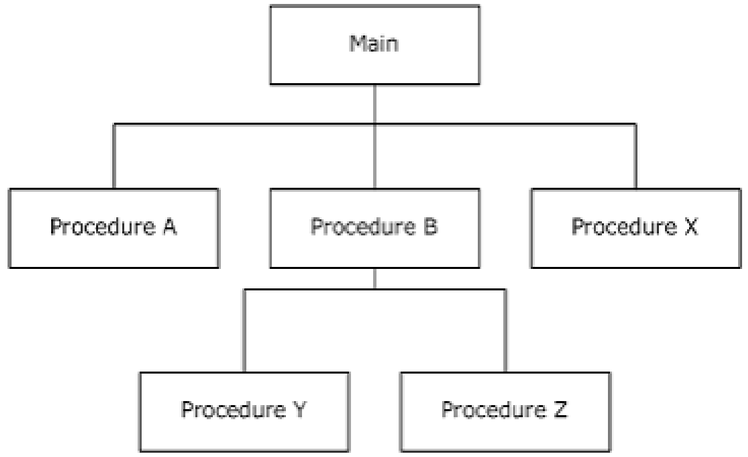
Examples: HTML, XML, LaTeX

Characteristics of Procedural Programming



- **Top down approach**

- It follows a **top-down approach**: you start with the big problem and break it down step by step.
- The program runs **sequentially** .
- The main program is divided down into smaller modules (functions or procedures)
- It uses global and local variables: some variables are accessible everywhere, while others are only available inside specific functions.
- Each module is then divided into sub modules,
- It is easy to understand and debug for small to medium-sized programs.



limitations of Procedural Programming (PP) :

- **Hard to manage large programs** :As the program grows---> the code becomes messy and difficult to follow.
- Poor Data Security (Lack of Encapsulation) : Data is not protected, as global variables can be accessed and modified from anywhere.
- Difficult to maintain and update – Changing one function can break other parts of the program because they are closely connected.
- Code Duplication – No inheritance means similar functions must be rewritten repeatedly.
- Scalability Issues --->More functions = more complexity. Large projects become hard to handle.



limitations of Procedural Programming (PP) :

- Procedural Programming **Does not model the real world well** :

  Real-life things are objects (like a car or person) with data and actions together.



Object Oriented Programming (was designed to overcome the limitations of procedural programming)

What is OOP?

Procedural programming focuses on writing functions that act on data.

Object-oriented programming focuses on creating objects that hold both data and the functions that work on that data.

OOP is a good way to write code that can:

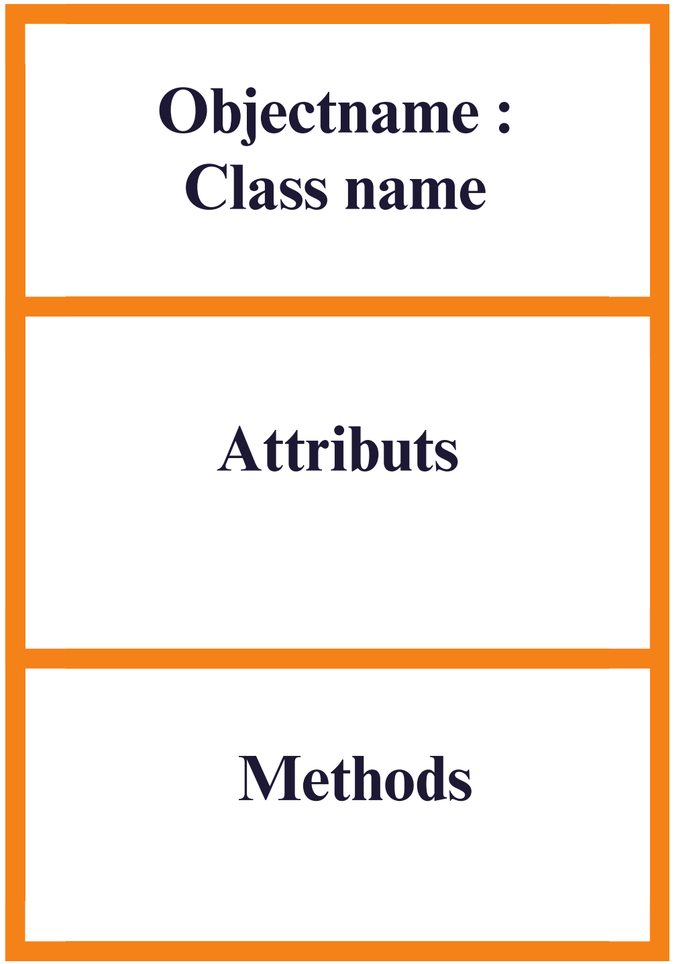
- scalable
- reusable
- maintainable

Object oriented programming has several advantages over procedural programming:

- OOP gives programs a clear and organized structure.
- OOP helps keep code DRY ("Don't Repeat Yourself"), which means less repetition. This makes code easier to maintain, change, and fix.
- OOP allows you to build fully reusable applications with less code and in less time.

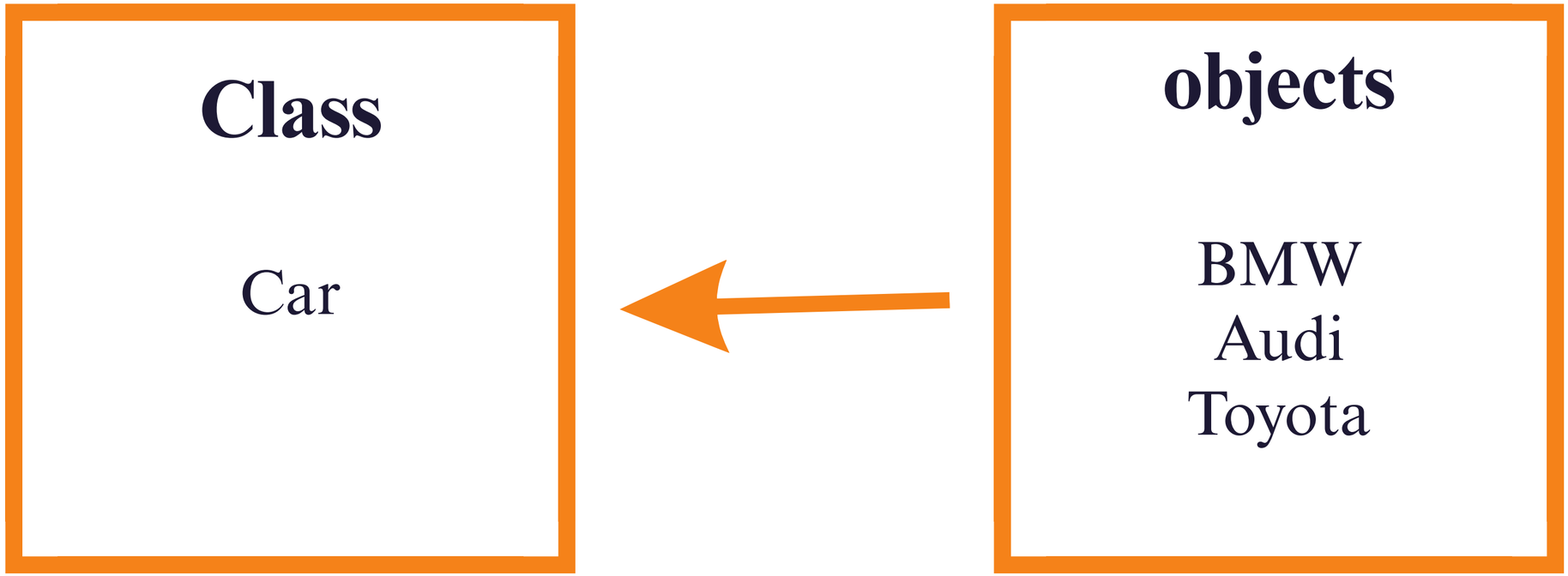
Characteristics of Oriented Object Programming

- Association of data structures and the processes that manipulate them into a coherent entities
- These entities are objects
- The data structures associated with an object are its attributes
- The processes associated with an object are its methods
- The attributes of an object are only accessible by its methods



What are Classes and Objects?

Classes and objects are the two main aspects of object-oriented programming.



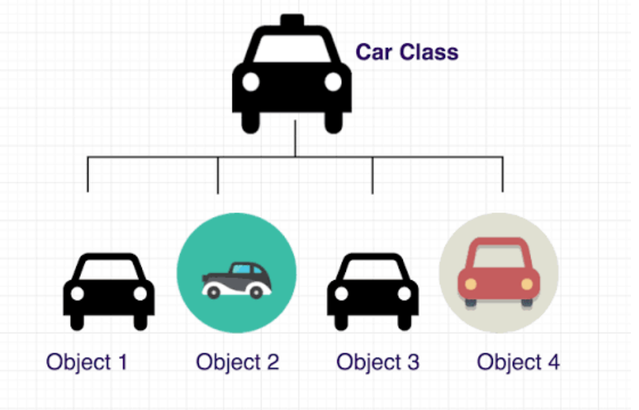
What are Classes and Objects?

Class

Car

objects

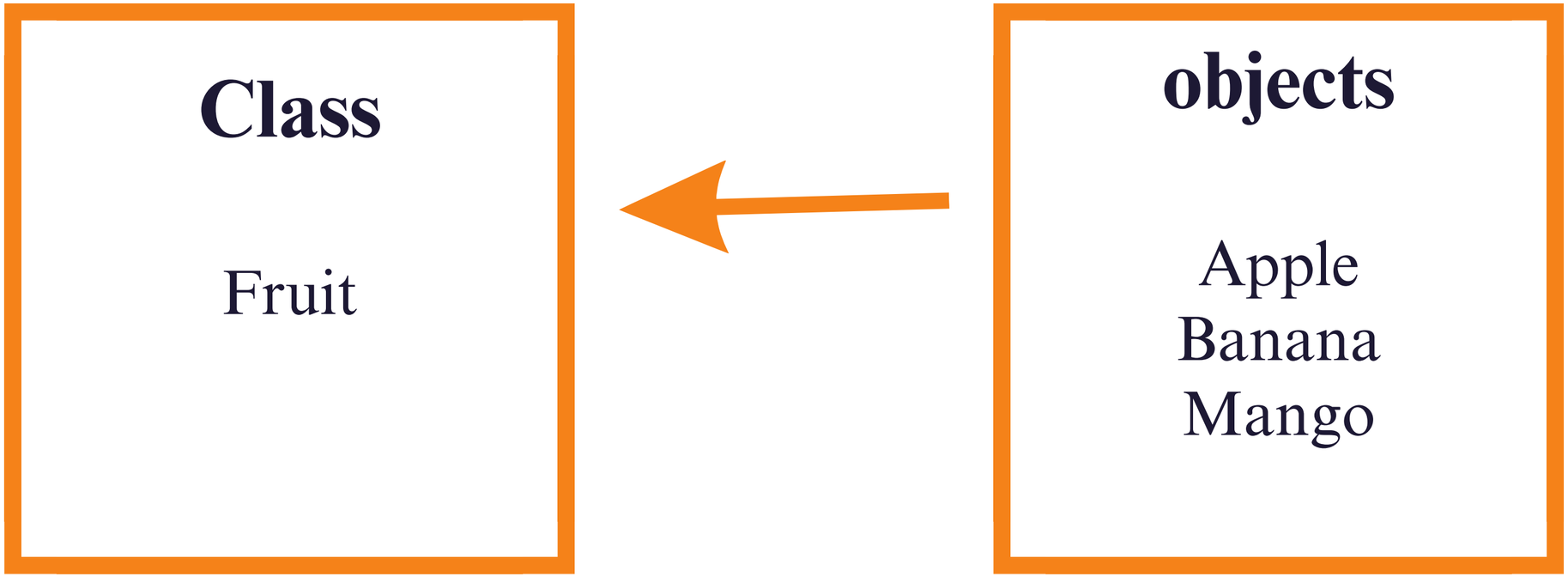
BMW
Audi
Toyota



A class is a template for objects, and an object is an instance of a class.

When the individual objects are created, they inherit all the variables and methods from the class.

What are Classes and Objects?



Key Concepts of Object-Oriented Programming

- **Class**

A class is a template or model that defines and describes what data and actions an object will have.

Attributes with their names and types (but not their values which are specific to each object).

methods consists of a signature (name + parameters) and the block of code that describes its action.

Object = an instance of a class.

Multiple objects can be made from one class – each has its own unique state.

Key Concepts of Object-Oriented Programming

- **Object**

An object is a coherent runtime entity that has

Attributes store the data that describes an object's current state.

Methods – functions that describe the behavior or actions of an object.

Objet = Identity + State (attributs) + Behavior methods

Key Concepts of Object-Oriented Programming

- **Message**

Objects interact by sending messages to one another.

A message is a request sent from one object to another, asking the receiving object to perform a specific action



A message represents the call of a method of the destination object by a source object

Key Concepts of Object-Oriented Programming

- ## Inheritance

Inheritance is a mechanism where one class (child/subclass) inherit the attributes and methods of another class (parent/superclass).

The "Is-A" rule:
Inheritance represents an "is-a" relationship. This means the child class is a specific type of the parent class.

For example :

- BMW is a car
- Cat is an animal

Key Concepts of Object-Oriented Programming:

- **Inheritance**

- **Inheritance**

The child class(subClass)

- Inherits all attributes and methods from the parent class
- Can add new attributes and methods
- Unique identity – each object has its own reference (memory address)
- Single inheritance : a class can inherit from only one parent class (in Java)

Parent class (superClass)

- Gives (provides) its attributes and methods to the child class.
- Does not receive anything back from the child
- Can have multiple subclass classes
- The superclass has the shared attributes and methods.

Key Concepts of Object-Oriented Programming:

- **Why Inheritance?**
  - Code Reusability – Child classes automatically get all attributes and methods from the parent
  - Reduce Redundancy – Common features are written once in the parent class and shared among all children, avoiding duplication.
  - Hierarchy – It creates a logical parent-child relationship between classes.
  - Easy Maintenance – Changes made in the parent class automatically apply to all children, so updates are easier.
  - Extensibility – Child classes can add new features or modify inherited ones, so you can build on existing code without changing it.

Key Concepts of Object-Oriented Programming:

- **Composition**

Composition is a strong "Has-A" relationship where one class “the whole” contains objects of another class “the part”



Key Concepts of Object-Oriented Programming:

- **Encapsulation**
  - Encapsulation consists of hiding part of attributes and methods and only allowing access through controlled methods
  - Hidden attributes are accessible by other objects through methods.





Key Concepts of Object-Oriented Programming:

- **Why Encapsulation**

  - Protects data from accidental changes – Internal variables cannot be modified directly from outside the object
  - Gives you full control – You decide what can be read, written, or hidden using getters and setters.
  - Makes maintenance easier
  - Provides a clear and consistent interface for interacting with the object.

Key Concepts of Object-Oriented Programming:

- **Encapsulation**

  Encapsulation allows to define visibility levels for attributes and methods.

  There are three levels of visibility:

- **public(+) :**
  - Any object can access attributes or methods marked as public.
  - This visibility level offers the least protection (it is the most open/access
- **private(-)**
  - only methods inside the same class can access this.
  - This offers the highest level of protection.
- **Protected(#):**
  - Access is only allowed for objects of child classes.

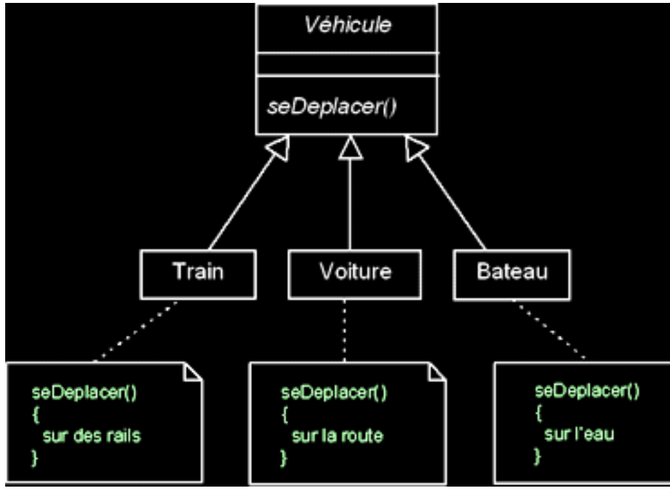
    “Same class + child/subclasses “

Key Concepts of Object-Oriented Programming:

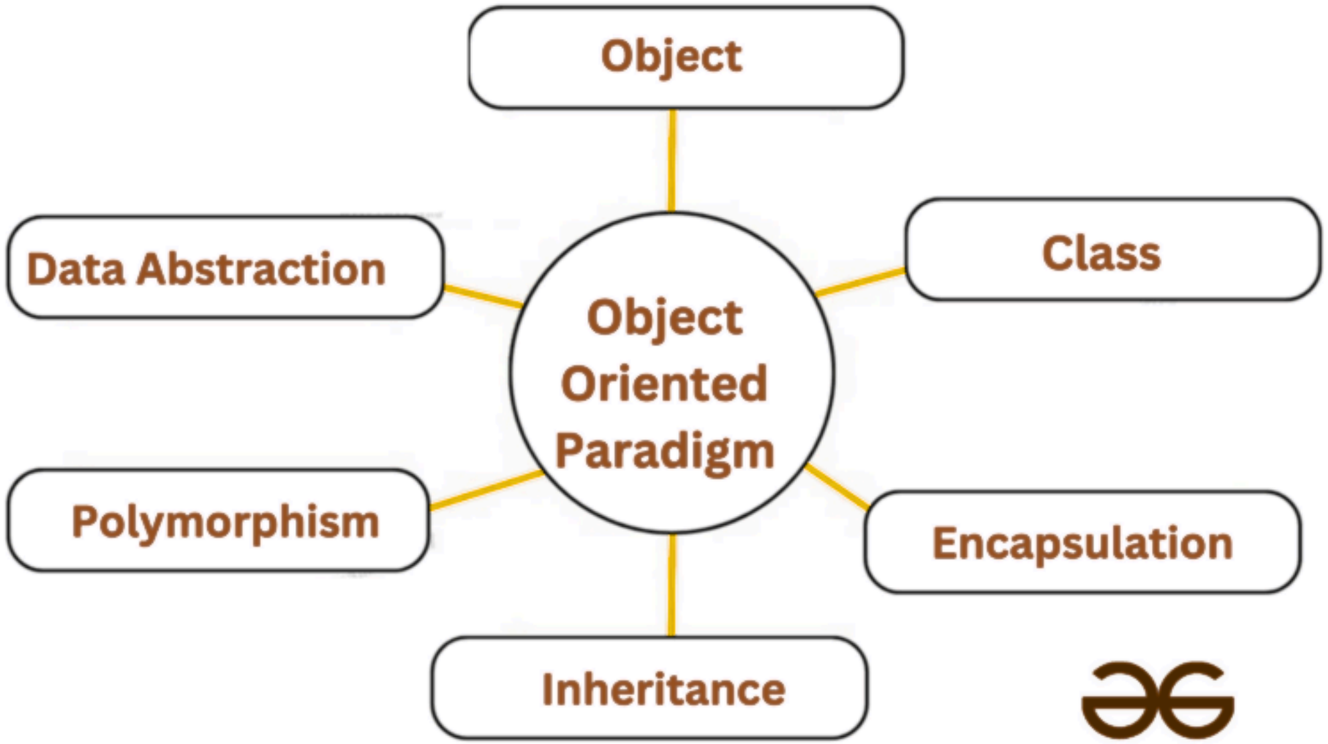
• Polymorphisme

In object-oriented programming, polymorphism means "many forms". It allows the same method or interface to have different behaviors depending on which object is calling it.

- Allows one interface to have multiple implementations making(Multi-Form Behavio)
- Polymorphism is the engine behind interfaces and abstract classes.
- Polymorphism makes the code more flexible and scalable
- It is achieved through method overloading and method overriding.



Key Concepts of Object-Oriented Programming





Introduction to Object-Oriented Programming using Java

Introduction to Object-Oriented Programming using Java



Source code file: .java

Compiled file : .class

Required tools: JRE, JDK, etc.

Environment used in lab sessions: NetBeans

Introduction to Object-Oriented Programming using Java



Java is a high-level, object-oriented programming language, created in 1995.

It is owned by Oracle, and more than 3 billion devices run Java.

Introduction to Object-Oriented Programming using Java



- Java is a widely used, open-source programming language. You can use it for many kinds of projects.
- It organizes everything using classes and objects, and it can handle multiple tasks.
- Platform Independence (WORA): Uses the "Write Once, Run Anywhere" principle—compiled bytecode runs across Windows, macOS, and Linux without modification via the Java Virtual Machine (JVM).
- It's easy to read and learn.

Introduction to Object-Oriented Programming using Java



- **It is used for:**
  - Mobile applications (specially Android apps)
  - Desktop applications
  - Web applications
  - Web servers and application servers
  - Games
  - Database connection, etc.

Introduction to Object-Oriented Programming using Java



- **Why Use Java?**
  - Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
  - It is one of the most popular programming languages in the world
  - It has a large demand in the current job market(Companies like Amazon, Netflix, and LinkedIn use Java)
  - It is easy to learn and simple to use
  - It is open-source and free
  - It is secure, fast and powerful

Introduction to Object-Oriented Programming using Java



- **Why Use Java?**
  - It has huge community support (tens of millions of developers)
  - Java is an object oriented language which gives a clear structure to programs.
  - As Java is close to C++ and C#, it makes it easy for programmers to switch to Java

Introduction to Object-Oriented Programming using Java



- **Example:**

  Hello World Program

- **Beginners usually start with the "Hello World" program when learning Java.**

Introduction to Object-Oriented Programming using Java



- **Example:**

```
public class HelloWorld                 // Defines a class named HelloWorld
{                                       // Opening brace for the class.

    public static void main(String[] args)    // The main method – entry point of any Java progr

   {  // Opening brace for the method

            System.out.println("Hello World!");   // Prints "Hello World!" to the console.

    } // Closing brace for the main method.
}   // Closing brace for the main method.
```

How to run the java code?



HelloWorld.java

Write code in a file like HelloWorld.java.

How to run the java code?



HelloWorld.java



Java Compilar

Java Compiler "javac" compiles it into bytecode "HelloWorld.class".

How to run the java code?



HelloWorld.java



Java Compilar



HelloWorld.class

How to run the java code?



HelloWorld.java



Java Compilar



HelloWorld..class

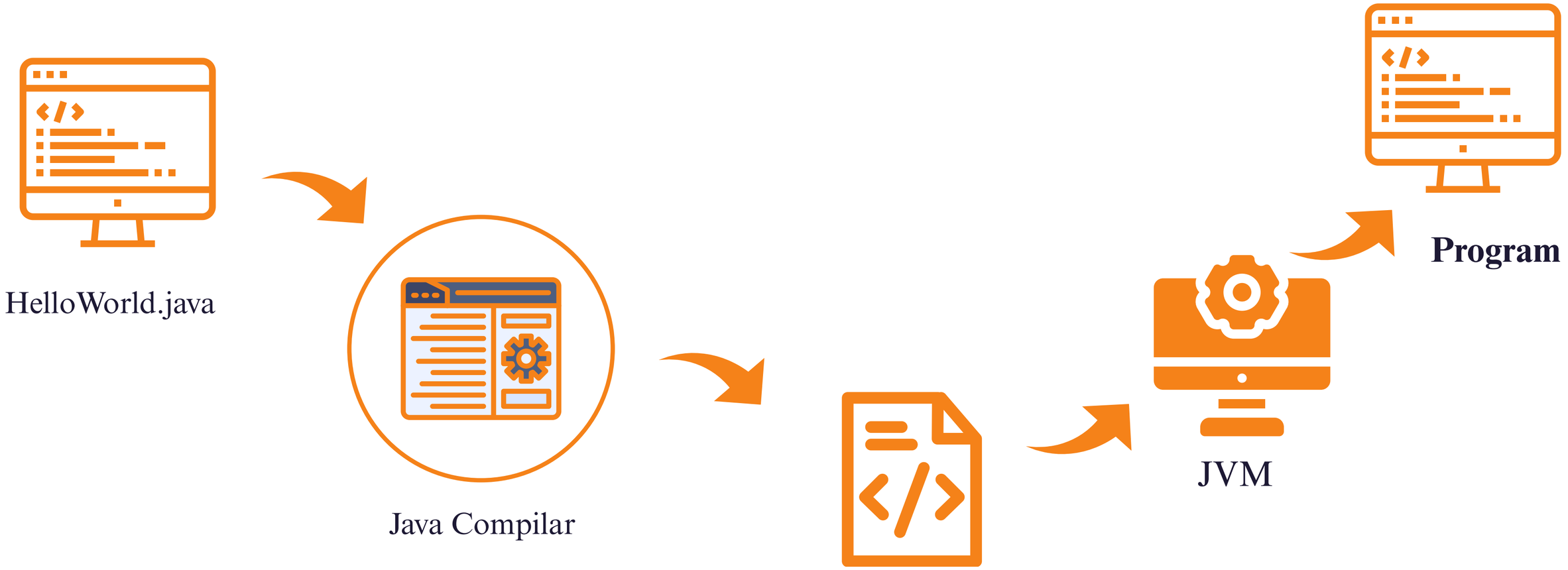
JVM (Java Virtual Machine) reads the .class file and interprets the bytecode



JVM

Java

How to run the java code?



Uses of Java



- Big Data Technologies use Java
- Android Apps
- Web apps
- Desktop GUI Apps
- With IoT, Java can be used on embedde
- Java is also widely used to build games such as: LibGDX and jMonkeyEngine.
- LinkedIn uses Java to manage large amounts of incoming traffic.
- Netflix uses Java to run its backend services and to handle content delivery
- Amazon uses Java to run its backend systems and keep its huge e-commerce platform working smoothly
- There are other big companies that use Java to build their services, such as Spotify, Uber, NASA WorldWind, and more.

https://www.geeksforgeeks.org/java/introduction-to-java/

Quiz:

What is used to compile Java source code?

- **A javac**
- **B java**
- **C jdk**
- **D jvm**

https://www.geeksforgeeks.org/java/introduction-to-java/

Quiz:

What is the correct way to declare a Java main method?

- **A public static void Main(String[] args)**
- **B public void main(String args)**
- **C public static void main(String[] args)**
- **D private static void main(String args[])**

https://www.geeksforgeeks.org/java/introduction-to-java/

Quiz:

Which of the following is NOT a feature of Java?

- **A Platform independence**
- **B Object-oriented**
- **C Dynamically typed language**

https://www.geeksforgeeks.org/java/introduction-to-java/

Quiz:

Which component converts Java bytecode into machine code?

- **A Java Compiler**
- **B JDK**
- **C JVM**
- **D JRE**

https://www.geeksforgeeks.org/java/introduction-to-java/



Java

